TASK: 13

Command Injection

&

Remote Code Execution (RCE)

Surendra Gabbani

(Command Injection)

What is Command Injection?

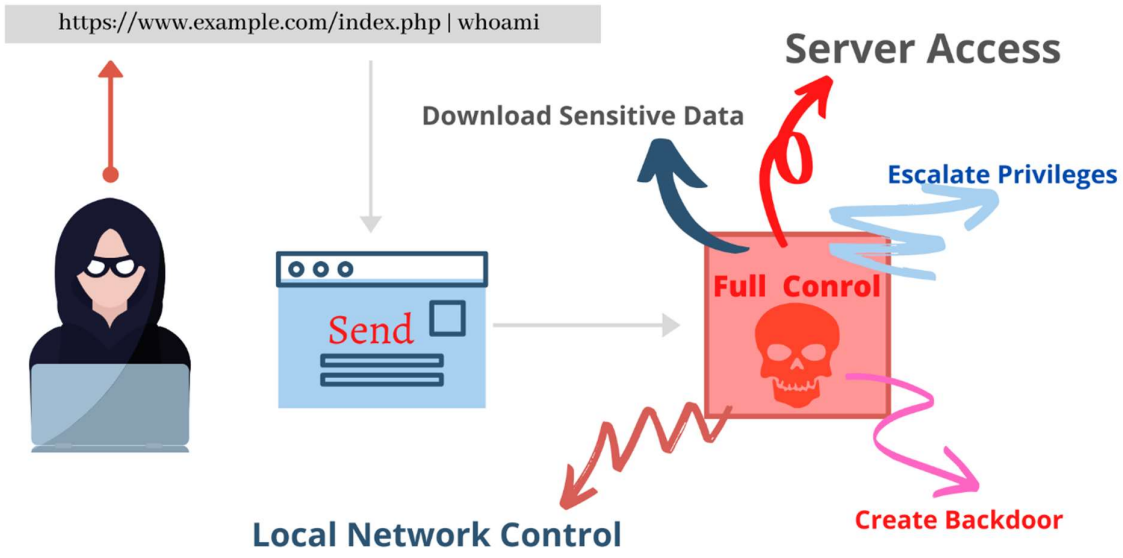
A vulnerability where an application passes unsafe user input to an operating-system command interpreter. successful exploitation allows attackers to run arbitrary commands on the host with the privileges of the vulnerable process.

How it works (attack surface):

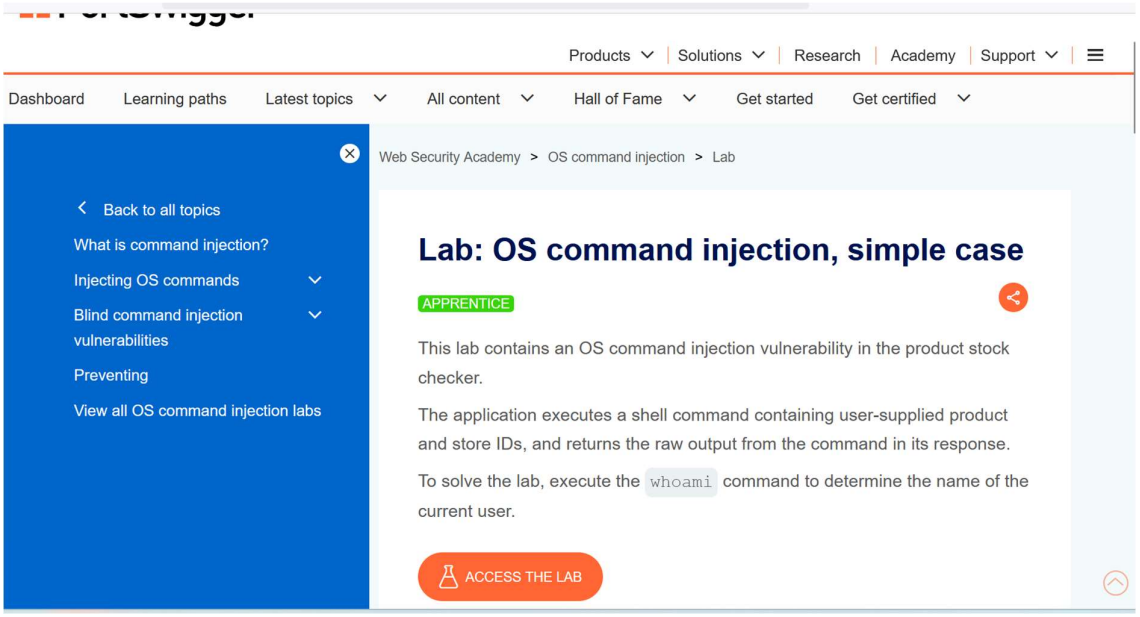
- Applications call out to the system shell (directly or indirectly).
- User-controlled data is concatenated into commands.
- Shell metacharacters (e.g., ;, &&, |) can chain extra commands if not handled.

Common risky patterns:

- Direct use of system(), popen(), backticks, exec() with concatenated strings.
- Use of high-level libraries that internally call a shell with shell=True or similar defaults.
- Passing user-controlled filenames, hostnames, or flags straight to shell commands.
- Relying only on client-side validation.

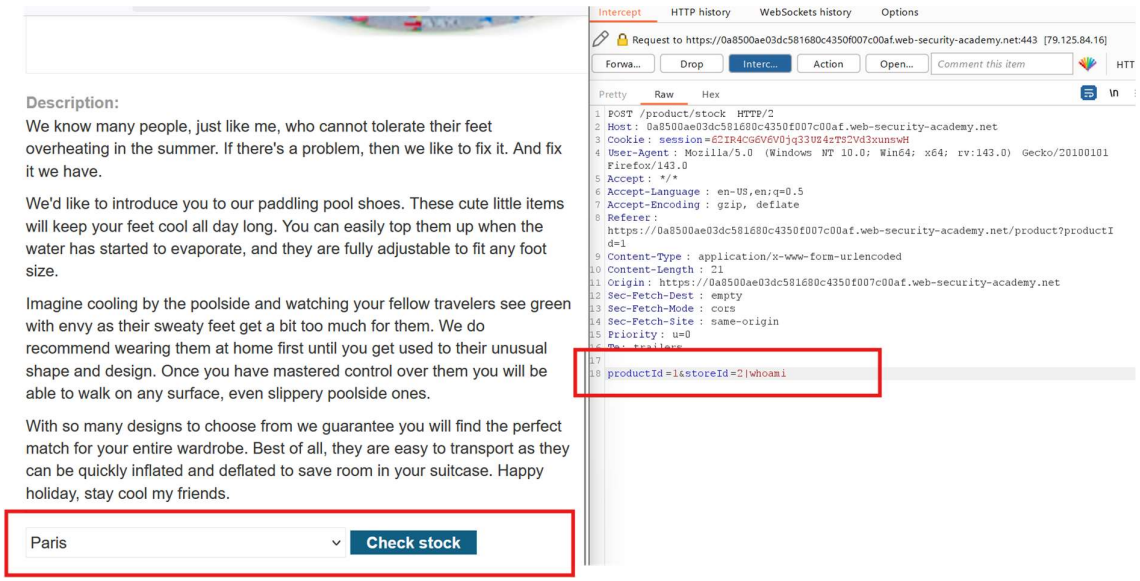


Lab & Practice -1



STEP 1:

- Turn on the burpsuite and interceptor ON and capture the request
- Inject the malicious command to the request



STEP 2: (solved)

Safe principles (how to prevent):

- Avoid the shell: prefer language-native APIs or libraries instead of calling OS tools.
- Use argument lists: if you must call a program, pass argv arrays (no shell) so the runtime doesn't parse metacharacters.
- Whitelist inputs: validate against allowed patterns (exact formats, enums, regex).
- Escape/quote: only when unavoidable, and use robust escaping libraries for that language/runtime.
- Least privilege & sandboxing: run command-invoking services with minimal rights and in isolated environments (containers, chroots).
- Audit + tests: add unit/CI tests for malformed inputs, static scanning for dangerous functions, and runtime monitoring of unexpected execs.

Defensive testing & detection:

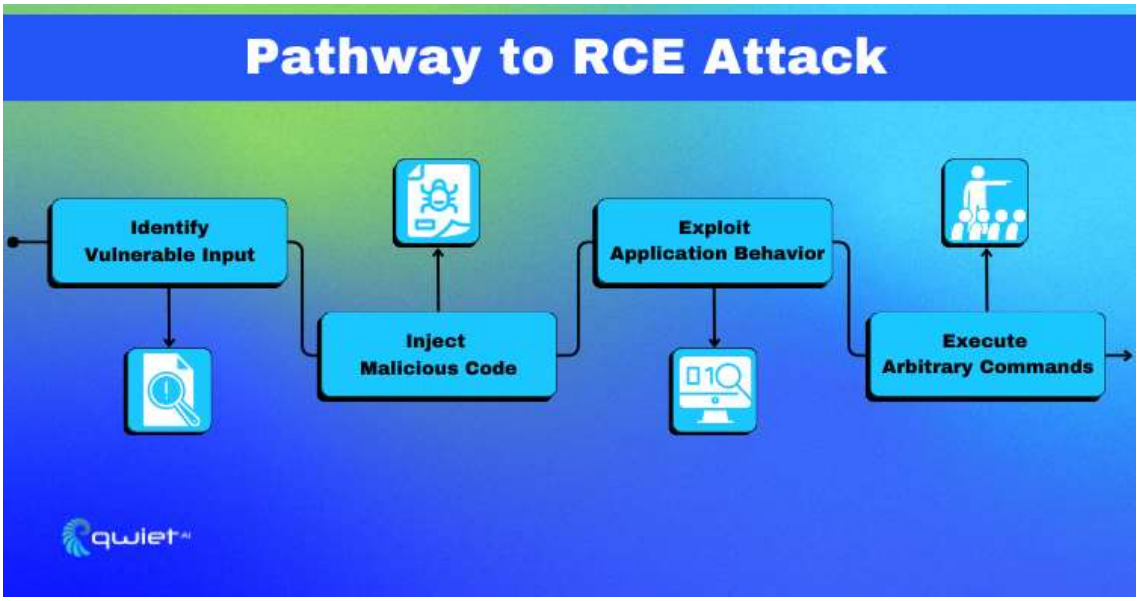
- Static code review: search for uses of system, exec, popen, os.system, backticks etc.
- Dynamic testing: fuzz inputs, use safe testing environments (isolated labs, containers).
- Logging & monitoring for anomalous command patterns and unexpected process executions.

(Remote Code Execution)

What RCE means?

Remote Code Execution (RCE) is a vulnerability that lets an attacker execute arbitrary code on a remote machine or server — without having direct physical or local access. It's one of the most severe vulnerability types because it often leads to full system compromise.

- Attacker can run commands, install backdoors, move laterally, exfiltrate data, or take complete control.
- Often gives the attacker the same privileges as the compromised process (which can be high).
- Frequently results in major breaches and high CVSS scores.



Common causes / attack vectors:

- Unsanitized input passed into eval() or equivalent.
- Template injection (server-side templates).
- Unsafe deserialization of untrusted data.
- File upload handlers executing files or allowing inclusion.
- Command injection via shell calls (e.g., system(), exec()).

LAB – 2

Lab: Remote code execution via web shell upload

APPRENTICE

LAB Solved

This lab contains a vulnerable image upload function. It doesn't perform any validation on the files users upload before storing them on the server's filesystem.

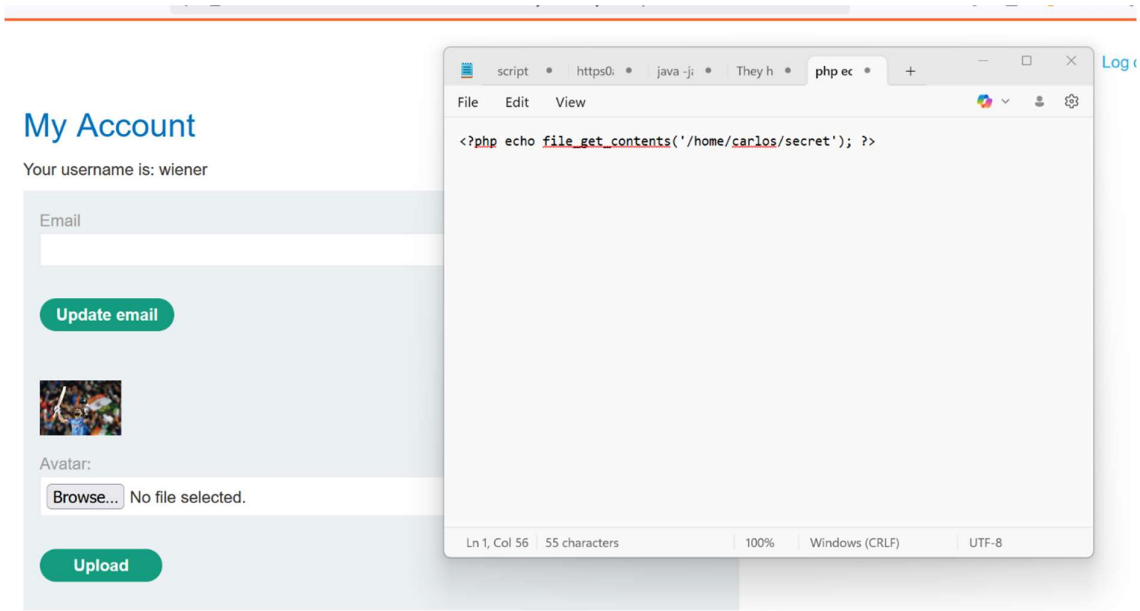
To solve the lab, upload a basic PHP web shell and use it to exfiltrate the contents of the file `/home/carlos/secret`. Submit this secret using the button provided in the lab banner.

You can log in to your own account using the following credentials: `wiener:peter`

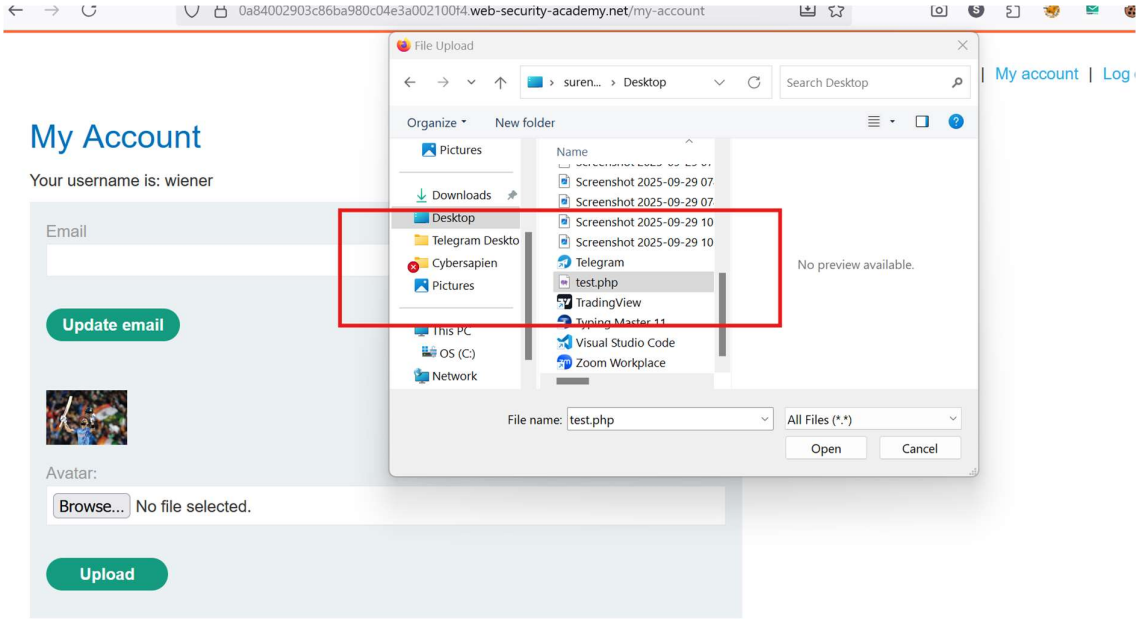
ACCESS THE LAB

STEP 1:

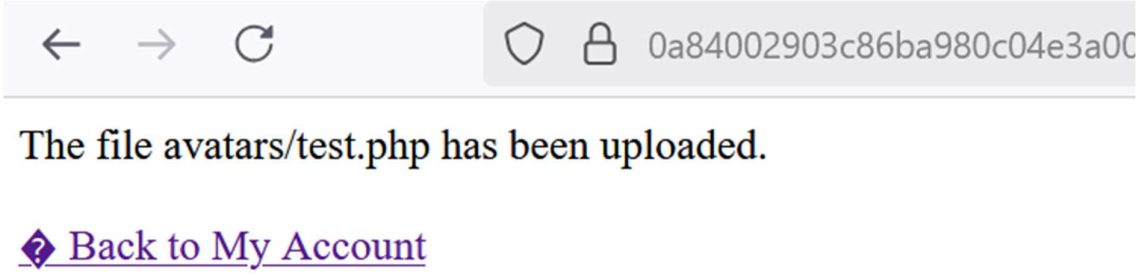
- Create a file called test.php, containing a script



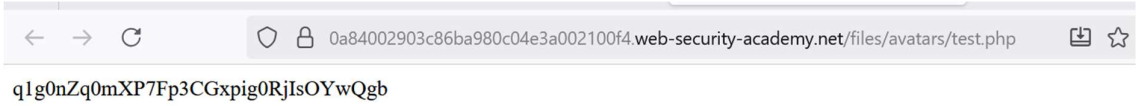
STEP 2: upload function to upload your malicious PHP file.



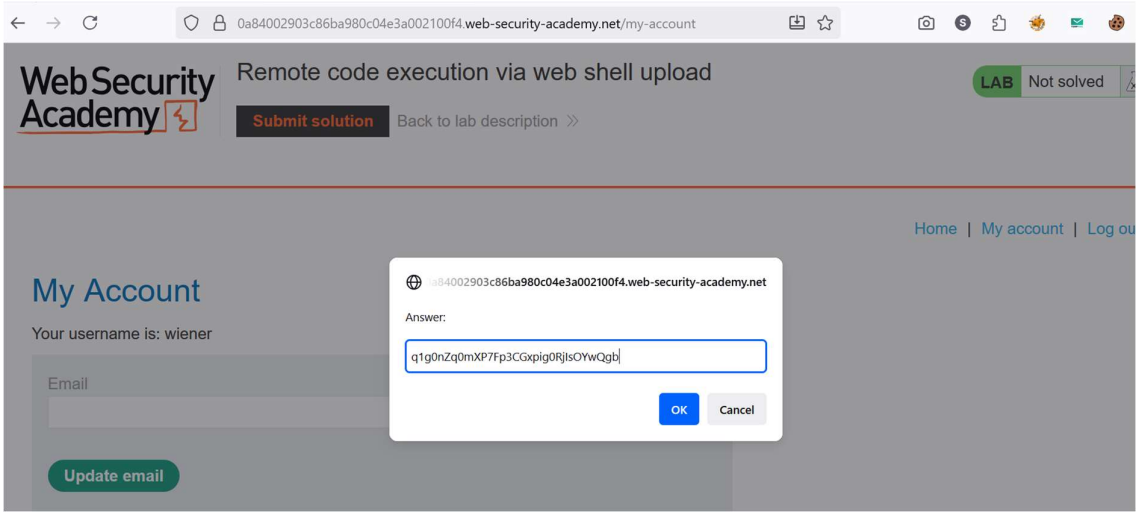
STEP3: after uploading that file, open in new tab

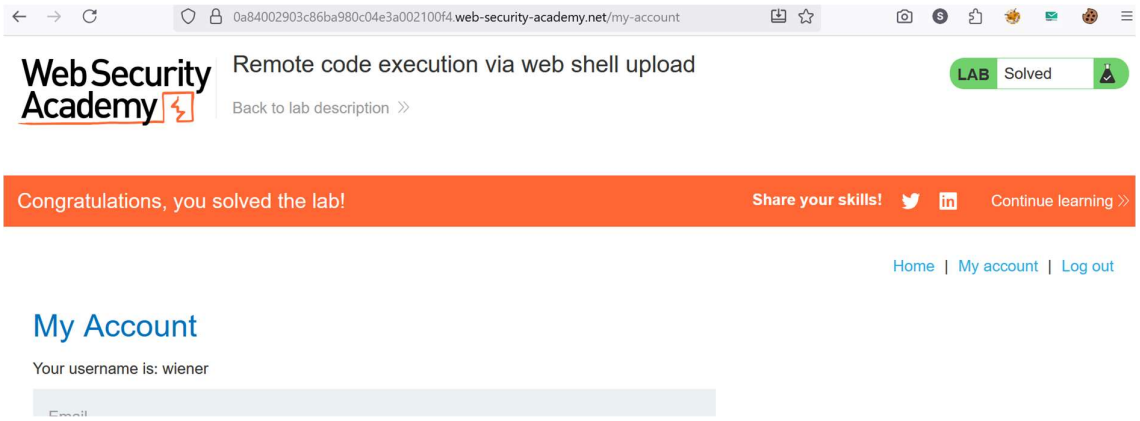


Need to upload this Key in submit solution



STEP 4: solved





Mitigations & best practices:

- Least privilege: run services with minimal permissions.
- Avoid dynamic code execution: remove eval() and similar where possible.
- Validate & sanitize inputs: employ whitelists, strict types.
- Safe deserialization: use allowlists or safer formats (JSON).
- Use secure libraries / patch promptly.
- Sandboxing & isolation: containers, seccomp, AppArmor, SELinux.
- Threat modeling & secure-by-design reviews.

Conclusion:

- Remote Code Execution (RCE) is one of the most critical security vulnerabilities because it allows attackers to run arbitrary code on a target system, often leading to full system compromise. Exploiting RCE can give attackers unauthorized access, data theft, privilege escalation, or even complete control over servers and applications.
- Preventing RCE requires secure coding practices, strict input validation, proper access control, timely patching, and continuous security testing. By combining proactive defenses with regular monitoring, organizations can reduce the risk of RCE attacks and protect sensitive systems and data.

THANK YOU